Módulo `settings`

> Código: `src/settings.h`, `src/settings.cpp` · Relacionados: 06 — Motor e NVS, main, posture

Papel no sistema

Guarda a configuração e a calibração na **NVS** (*Non-Volatile Storage*), para sobreviverem a desligar a placa. O ESP32 não tem EEPROM: a NVS é uma área da flash organizada em pares chave/valor, acessada pela biblioteca `Preferences` do core Arduino. Também define os tipos `Config`, `Calibracao` e `ModoSensor`, usados pelo resto do firmware.

Este módulo **não guarda** os valores em uso: quem guarda é `main.cpp` (`cfg`, `cal`). Aqui só se lê (no boot) e grava (quando algo muda).

Dependências

- Usa: `Preferences` (core Arduino-ESP32; no simulador, `sim/Preferences.h` guarda em memória), posture (`Params`, `paramsPadrao`, `validarParams`), tilt (`Vec3`, `v3`, `norm`).
- É usado por: `main.cpp` (carregar no boot, salvar config e calibração), `console.cpp` e `protocolo.cpp` (`configPadrao`, `modoNome`, `modoDeTexto`).
- Roda em: `settings_carregar` no `setup`; `settings_salvarConfig` no loop (via `app_definirConfig`); `settings_salvarCalibracao` na tarefa de controle (via `concluirCalibracao`).

Tipos e constantes

`enum ModoSensor : uint8_t`

Valor	Número	Texto	Significado
`MODO_AUTO`	0	`auto`	usa o sensor se ele for detectado (e tiver referência)
`MODO_ON`	1	`on`	força ligado (precisa de referência da calibração)
`MODO_OFF`	2	`off`	nunca usa

O número vai para a NVS: não reordenar o enum.

`struct Config`

Campo	Tipo	Descrição
`params`	`Params`	limite, histerese, tempo mínimo, limite do flex, intensidade (posture)
`flexModo`	`ModoSensor`	flex 1 (GPIO34); padrão `auto`
`flex2Modo`	`ModoSensor`	flex 2 (GPIO35); padrão `off`

`struct Calibracao`

Campo	Tipo	Unidade	Descrição
`valida`	bool	—	`false` = nunca calibrado (o colete não vibra)
`g0`	`Vec3`	unitário	vetor gravidade na postura correta (a referência do desvio)
`flex0`	float	ADC	leitura do flex 1 na postura correta (0 = sem referência)
`flex20`	float	ADC	leitura do flex 2 na postura correta (0 = sem referência)

Chaves na NVS (namespace `colete`)

Chave	Tipo	Campo	Padrão (chave ausente)
`limite`	float	`params.limiteGraus`	`PADRAO_LIMITE_GRAUS` (20)
`hist`	float	`params.histerese`	`PADRAO_HISTERESE` (3)
`tempo`	uint32	`params.tempoMs`	`PADRAO_TEMPO_MS` (3000)
`flexlim`	float	`params.limiteFlex`	`PADRAO_LIMITE_FLEX` (15)
`motor`	uint8	`params.intensidade`	`PADRAO_INTENSIDADE` (100)
`flexmodo`	uint8	`flexModo`	0 (`auto`)
`flex2modo`	uint8	`flex2Modo`	2 (`off`)
`cal_ok`	bool	`cal.valida`	`false`
`g0x`, `g0y`, `g0z`	float	`cal.g0`	(0, 1, 0)
`flex0`	float	`cal.flex0`	0
`flex20`	float	`cal.flex20`	0

A NVS aceita nomes de chave de no máximo 15 caracteres; por isso os nomes são curtos.

Funções

`const char* modoNome(ModoSensor m)`

- **O que faz:** `MODO_ON` → `"on"`, `MODO_OFF` → `"off"`, qualquer outro → `"auto"`.
- **Chamada por:** `console.cpp` (`get`), `protocolo.cpp` (`{"t":"config"}`).

`bool modoDeTexto(const char* txt, ModoSensor* m)`

- **O que faz:** converte texto em modo. Aceita `on`/`1`/`true`, `off`/`0`/`false` e `auto` (minúsculas; a serial já converte para minúsculas antes).
- **Retorno:** `false` se o texto não for reconhecido; nesse caso `*m` não é alterado.
- **Chamada por:** `console.cpp` (`set flex ...`), `protocolo.cpp` (`flexModo`/`flex2Modo` do `set`).

`Config configPadrao()`

- **O que faz:** valores de fábrica: `paramsPadrao()`, flex 1 `auto`, flex 2 `off`.
- **Por que o flex 2 vem desligado:** um GPIO35 solto "copia" a tensão do GPIO34 pelo capacitor de amostragem do ADC e pareceria um sensor de verdade. O 2º flex só entra se for ligado de propósito (`set flex2 auto` ou `on`) e depois de recalibrar.
- **Chamada por:** `settings_carregar` (padrões), `console.cpp` (`reset`), `protocolo.cpp` (`reset`).

`void settings_carregar(Config* c, Calibracao* cal)`

- **O que faz:** lê todas as chaves (só leitura). Chave ausente = padrão. Depois corrige o que vier estranho da flash:
  - `validarParams` nos parâmetros (mesmas faixas da serial/app);
  - modo fora de 0–2 vira `auto`;
  - calibração com `|g0| < 0,5` é descartada (`valida = false`).
- **Chamada por:** `setup` (uma vez, no boot).

`void settings_salvarConfig(const Config& c)`

- **O que faz:** grava as 7 chaves da configuração (todas, mesmo as que não mudaram).

- **Chamada por:** `app_definirConfig` (loop).

`void settings_salvarCalibracao(const Calibracao& cal)`

- **O que faz:** grava as 6 chaves da calibração. Não mexe na configuração.
- **Chamada por:** `concluirCalibracao` (tarefa de controle).

Armadilhas e cuidados

- **Renomear uma chave = perder o valor salvo** nas placas já gravadas (volta ao padrão).
- **`reset` não apaga a calibração:** são chaves diferentes; `reset` só regrava a configuração.
- **Modo inválido na flash vira `auto`, inclusive no flex 2** (cujo padrão é `off`). Só acontece com dado corrompido, mas nesse caso o GPIO35 solto poderia ser detectado como flex; ele ainda precisaria de uma referência válida da calibração para entrar na detecção.
- Para zerar tudo (inclusive a calibração), é preciso apagar a flash (`pio run -t erase`) e gravar de novo.
- No simulador a NVS vive só na memória do processo.